

B. Proofs

Theorem 1. *Let n be the number of queries running in FunShare, Algorithm 2 produces a set of sharing groups within at most n executions. Also, in the absence of backpressure, existing sharing groups are not affected.*

Proof. We prove this theorem by induction:

- $n = 1$: The initial query group consists by one query, so it is by definition a sharing group.
- Suppose that the property holds for $n \leq k$, we prove that it also holds for $n = k + 1$:

Let m be the number of groups across which the queries are partitioned. Furthermore, let PQ be the penalized queries and BQ the queries experiencing high backpressure. There are three cases:

- 1) $m = 1$ and $PQ \cup BQ = \emptyset$: then by definition, the group is already a sharing group and it remains as is, i.e. unaffected by the algorithm.
- 2) $m = 1$ and $BQ \cup PQ \neq \emptyset$: if $BQ \neq \emptyset$, Algorithm 2 forms singleton groups for queries in BQ , else for queries in PQ . Either way, let $RQ = BQ$, if $BQ \neq \emptyset$, else PQ , the algorithm forms singleton groups for queries in RQ which are sharing groups. Furthermore, $|g_1 - RQ| \leq k$ so running Algorithm 2 for at most k more iterations will result to sharing groups for the queries in this group main group.
- 3) $m \geq 2$: Algorithm 2 runs independently across each group. Each $|g_j| \leq k$ so running the algorithm for each group independently leads to sharing groups within at most $\sum_{j=1}^m |g_j| = k + 1$ steps. If any group is already a sharing group and $BQ = \emptyset$, it is not affected.

□

Theorem 2. *Let m be the number of groups in FunShare. Assuming an accurate model for $\text{Load}(g)$, linear scalability for operators and a merging threshold that is at most 1, Algorithm 1 has the following loop invariant: if the m groups are all sharing groups and queries with backpressure are isolated at the start of the loop, they remain so at the end of the loop as well.*

Note that an accurate model for $\text{Load}(g)$ and linear scalability are not always possible. For this reason, we set lower, more pessimistic values for the merging threshold, in order to compensate for estimation errors in our model.

Proof. In each iteration of Algorithm 1's loop, since $\text{MERGING_THRESHOLD} < 1$, the algorithm chooses a pair of groups such that

$$\max(\text{GroupingCost}(g_i, g_j), \text{GroupingCost}(g_j, g_i)) < 1$$

We note that, due to the assumption of linear scalability, each group's throughput is

$$T_g = \frac{\text{Resources}(g) - \text{IdleResources}(g)}{\text{Load}(g)}$$

and the maximum sustainable throughput T^* for $g_i \cup g_j$ is

$$T^* = \frac{\text{Resources}(g_i) + \text{Resources}(g_j)}{\text{Load}(g_i \cup g_j)}$$

We have

$$\text{GroupingCost}(g_i, g_j) < 1 \implies$$

$$\frac{\text{Resources}(g_i) + \text{Resources}(g_j)}{\text{Load}(g_i \cup g_j)} \leq \frac{\text{Resources}(g_i) + \text{IdleResources}(g_j)}{\text{Load}(g_i) - \text{Load}(g_i \cap g_j)}$$

therefore

$$T^* = \frac{\text{Resources}(g_i) + \text{Resources}(g_j)}{\text{Load}(g_i \cup g_j)} \implies$$

$$T^* = \frac{\text{Resources}(g_i) + \text{IdleResources}(g_j)}{\text{Load}(g_i \cup g_j)} + \frac{\text{Resources}(g_j) - \text{IdleResources}(g_j)}{\text{Load}(g_i \cup g_j)} \implies$$

$$T^* = \frac{\text{Resources}(g_i) + \text{IdleResources}(g_j)}{\text{Load}(g_i \cup g_j)} \frac{\text{Load}(g_i) - \text{Load}(g_i \cap g_j)}{\text{Load}(g_i) - \text{Load}(g_i \cap g_j)} + \frac{\text{Resources}(g_j) - \text{IdleResources}(g_j)}{\text{Load}(g_i \cup g_j)}$$

But $\text{Load}(g_i \cup g_j) > \text{Load}(g_i) - \text{Load}(g_i \cap g_j)$ so

$$T^* \geq \frac{\text{Resources}(g_i) + \text{Resources}(g_j)}{\text{Load}(g_i \cup g_j)} \frac{\text{Load}(g_i) - \text{Load}(g_i \cap g_j)}{\text{Load}(g_i \cup g_j)} + \frac{\text{Resources}(g_j) - \text{IdleResources}(g_j)}{\text{Load}(g_i \cup g_j)} \implies$$

$$T^* \geq T^* \frac{\text{Load}(g_i) - \text{Load}(g_i \cap g_j)}{\text{Load}(g_i \cup g_j)} + \frac{\text{Resources}(g_j) - \text{IdleResources}(g_j)}{\text{Load}(g_i \cup g_j)} \implies$$

$$\frac{\text{Load}(g_i \cup g_j) - \text{Load}(g_i) + \text{Load}(g_i \cap g_j)}{\text{Load}(g_i \cup g_j)} T^* \geq \frac{\text{Resources}(g_j) - \text{IdleResources}(g_j)}{\text{Load}(g_i \cup g_j)} \implies$$

$$\frac{Load(g_j)}{Load(g_i \cup g_j)} T^* \geq \frac{Resources(g_j) - IdleResources(g_j)}{Load(g_i \cup g_j)} \implies$$

$$T^* \geq \frac{Resources(g_j) - IdleResources(g_j)}{Load(g_j)} \implies T^* \geq T_{g_j}$$

Similarly we can prove that

$$T^* \geq T_{g_i}$$

from $GroupingCost(g_j, g_i) < 1$

From these equations, we conclude that the new sharing group can sustain a higher throughput hence maintain functional isolation. Therefore, the resulting group set from the merging step comprises sharing groups. $\square$

The implication of the two theorems is as follows: the first time Algorithm 2 runs, it produces sharing groups. Then, Algorithm 1 performs merging iterations until it reaches a fixed point. From that point on, neither Algorithm changes the sharing groups, so FunShare has converged into a grouping. Note that convergence assumes that the data distribution doesn't change during this process; if so, repartitioning the queries is not only necessary but also desirable.